DOM Manipulations

In JavaScript, the **DOM (Document Object Model)** is a programming interface that allows scripts to access and manipulate the structure, style, and content of HTML documents. It represents the page as a tree of objects, where each node corresponds to a part of the document (such as an element, attribute, or text content).

Types in Dom manipulations:

Element accessing

↓

→getElementById()

→getElementsByTagName()

→getElementsByClassName()

→querySelector()

→querySelectorAll()

Element Manipulations

↓

→appendChild()

→removeChild()

→replaceChild()

→insertBefore()

→CloneNode()

Attribute Manipulation

↓

→getAttribute()

→setAttribute()

→removeAttribute()

→hasAttribute()

Event Handling

↓

→addEventListener()

→removeEventListener()

→dispatchEvent()

CSS manipulation

↓

→classList.add()

→classList.remove()

→classList.containes()

→classList.toggle()

Element Creation

↓

→createElement

→createTextNode()

→createDocumentFragment()

Element accessing

1. getElementById(): getElementById() is a widely used method in JavaScript for DOM (Document Object Model) manipulations. It allows you to select and manipulate HTML elements by their id attribute. Here's how it works and how you can use it:

Syntax:

```
document.getElementById('id')
```

Ex:

```
<body>
     <h1 id="id1"></h1>

   <script>
       document.getElementById("id1").inne
rHTML = "Welcome to ExcelR";

   <script>
<body>
```

- "Accessing the paragraph and add content using innerHTML as follows":

```
<body>
      <p id="id2"></p>

   <script>

       document.getElementById("id2").inn
erHTML = "Hello All.. This is Java Full
Stack Class...!!!"

   <script>
<body>
```

2)getElementsByTagName():

getElementsByTagName() is a method in JavaScript used for DOM (Document Object Model) manipulations. It allows you to select elements by their tag name (e.g., div, p, span). Unlike getElementById(), which returns a single element, getElementsByTagName() returns a live HTMLCollection of all elements with the specified tag name.

Syntax:

```
document.getElementsByTagName('tagName')
```

```
  <body>
        <h1></h1>
        <h1></h1>
        <h2></h2>
        <h2></h2>
        <p></p>
        <p></p>
        <h1></h1>
     <script>
   let all_h1 =
document.getElementsByTagName("h1");

        for(let h1 of all_h1){
            h1.innerHTML = "First h1,
second h1, third h1";
        }
      </script>
</body>
```

- Accessing all "h2" elements

```
<script>
let all_h2 =
document.getElementsByTagName("h2");

        for(let h2 of all_h2){
            h2.innerHTML = "First h2,
second h2, third h2";
        }
</script>
```

- Accessing all paragraphs

```
<script>
let all_p =
document.getElementsByTagName("p");

        for(let p of all_p){
            p.innerHTML = "First p,
second p, third p";
        }
</script>
```

3)getElementsByClassName():

getElementsByClassName() is a method in JavaScript used for DOM (Document Object Model) manipulations. It allows you to select all elements that have a specific class name. Like getElementsByTagName(), this method returns a live HTMLCollection of elements that match the specified class name.

Syntax:

```
document.getElementsByClassName('className');
```

```
   <body>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
   <script>
      let all_p =
document.getElementsByClassName("c1");

        for(let p of all_p) {
            p.innerHTML = "Hello";
            p.style.color = "red";
        }
   </script>
 <body>
```

- Accessing all paragraphs with the class "C2".

```
<body>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
   <script>
      let all_p =
document.getElementsByClassName("c2");

        for(let p of all_p) {
            p.innerHTML = "Welcome";
            p.style.color = "green";
        }
   </script>

 <body>
```

4) querySelector(): querySelector() is a powerful and versatile method in JavaScript used for DOM (Document Object Model) manipulations. It allows you to select the first element that matches a specified CSS selector within the document.

Syntax:

```
document.querySelector('selector');
```

```
<body>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
   <script>
      let p =
document.querySelector(".c1");

        p.innerHTML = "c1...I am
getting by querySelector()";
   </script>

 <body>
```

- Accessing first paragraph with the class "C2".

```
<body>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
   <script>
let p = document.querySelector(".c2");
        p.innerHTML = "c2...I am
getting by querySelector()";
   </script>

 <body>
```

5) querySelectorAll(): querySelectorAll() is a method in JavaScript used for DOM (Document Object Model) manipulations. It allows you to select all elements that match a specified CSS selector and returns a static NodeList of those elements. This method is similar to querySelector(), but instead of returning just the first match, it returns all matches.

Syntax:

```
document.querySelectorAll('selector');
```

```
<body>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
   <script>
let all_p =
document.querySelectorAll(".c1");
        for(let p of all_p) {
            p.innerHTML = "Hello";
            p.style.color = "red";
        }
   </script>

 <body>
```

- Accessing all paragraphs with the class "C2".

```
<body>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
        <p class="c1"></p>
        <p class="c2"></p>
   <script>
let all_p =
document.querySelectorAll(".c2");
        for(let p of all_p) {
            p.innerHTML = "Welcome";
            p.style.color = "green";
        }
   </script>

 <body>
```

Element Creation

1) createElement(): In DOM (Document Object Model) manipulations, createElement() is a method provided by the document object in JavaScript. It is used to create a new HTML element dynamically. This method doesn't add the newly created element to the DOM tree; it just creates it in memory. You can later add this element to the DOM by using methods like appendChild(), insertBefore(), or similar.

Syntax:

```
let newElement =
document.createElement('tagName');
```

2) createTextNode(): createTextNode() is a method in the DOM (Document Object Model) used to create a new text node in JavaScript. Unlike createElement(), which creates HTML elements, createTextNode() specifically creates a text node containing the text you specify.

Syntax:

```
let textNode =
document.createTextNode('text content');
```

3)createDocumentFragment():createDocumentFragment() is a method in the DOM (Document Object Model) that creates a minimal, lightweight container known as a document fragment. A document fragment is a special type of node that serves as a temporary storage container for other DOM nodes.

Syntax:

```
let fragment =
document.createDocumentFragment();
```

Element Manipulations

1) appendChild():appendChild() is a method in the DOM (Document Object Model) used to add a node as the last child of a specified parent node. This method is often used to insert new elements or nodes into the DOM, whether they were created dynamically with methods like createElement() or createTextNode() or if they were already existing nodes that need to be moved or restructured.

Syntax:

```
parentNode.appendChild(childNode);
```

```
<body>
        <div id="id1"></div>

<script>
    let h1 =
document.createElement("h1");
        h1.textContent = "ExcelR";
        document.getElementById("id1").
appendChild(h1);

</script>

 <body>
```

Q) What are the differences between innerHTML and textContent?

a) innerHTML and textContent are two properties in the DOM (Document Object Model) used to manipulate and retrieve content within HTML elements, but they function quite differently and have distinct use cases.

- innerHTML: innerHTML is used to get or set the HTML content of an element, including any HTML tags and nested elements.
- textContent: textContent is used to get or set the textual content of an element, ignoring any HTML tags.

2) removeChild(): removeChild() is a method in the DOM (Document Object Model) used to remove a specified child node from its parent node. This method is commonly used when you want to delete an element from the DOM, either to clean up the document or to dynamically modify the structure of a webpage.

Syntax:

```
parentNode.removeChild(childNode);
```

```
<body>
        <div id="id1">

        <h1 id="my_h1">Hello</h1>

        <h2 id="my_h2">Java</h2>
        </div>
```

```
<script>

let h2 =
document.getElementById("my_h2");
let div =
document.getElementById("id1");
div.removeChild(h2);
</script>

</body>
```

3) replaceChild():replaceChild() is a method in the DOM (Document Object Model) used to replace an existing child node of a parent node with a new node. This method is useful when you want to swap out one element for another in the DOM, effectively removing the old node and inserting the new one in its place.

Syntax:

```
parentNode.replaceChild(newChild, oldChild);
```

```
<body>
        <div id="id1">

          <h2 id="my_h2">ExcelR</h2>
        </div>
<script>

let p = document.createElement("p");
        p.innerHTML = "Welcome";
        let div =
document.getElementById("id1");
        let h2 =
document.getElementById("my_h2");
        div.replaceChild(p,h2);
</script>

</body>
```

4)insertBefore(): insertBefore() is a method in the DOM (Document Object Model) used to insert a new node into the DOM tree before a specified existing child node of a parent node. This method allows you to insert new elements or nodes at a specific position within the children of a parent element, rather than simply appending them to the end.

Syntax:

```
parentNode.insertBefore(newNode,
referenceNode);
```

```
<body>
   <div id="id1">

       <p id="my_p">Hello, ExcelR</p>
   </div>
<script>

let h6 = document.createElement("h6");
   h6.innerHTML = "Hello, DOM !!!";
    let p =
document.getElementById("my_p");
        let div =
document.getElementById("id1");
        div.insertBefore(h6,p);
</script>
</body>
```

5) cloneNode():cloneNode() is a method in the DOM (Document Object Model) that creates a copy of a specified node. This method can be used to duplicate an element and optionally, its entire subtree (i.e., all of its child nodes). The cloned node is not automatically inserted into the document; it needs to be appended to a parent node to become part of the DOM.

Syntax:

```
let clone = originalNode.cloneNode(true);
```

```
<body>
        <div id="id1">

            <h1 id="my_h1">Hello</h1>
        </div>
<script>

    let h1 =
document.getElementById("my_h1");
 let xerox_h1 = h1.cloneNode(true);
 let div =
document.getElementById("id1");
        div.appendChild(xerox_h1);
</script>

</body>
```

Attribute Manipulation

1) getAttribute(): getAttribute() is a method in the DOM (Document Object Model) that is used to retrieve the value of a specified attribute from an HTML element. This method is useful when you need to access custom data attributes, or standard attributes like id, class, href, src, etc., of an element.

Syntax:

```
let attributeValue =
element.getAttribute(attributeName);
```

```
<body>
    <p id="my_p">Hello, ExcelR</p>

<script>

   let p =
document.getElementById("my_p");

document.write(p.getAttribute("id"));
</script>
</body>
```

2) setAttribute():setAttribute() is a method in the DOM (Document Object Model) used to set or change the value of an attribute on an HTML element. This method allows you to add new attributes or update existing ones on an element.

Syntax:

```
element.setAttribute(attributeName,
attributeValue);
```

```
<body>
    <p id="my_p">Hello, ExcelR</p>

<script>

  let p =
document.getElementById("my_p");

   p.setAttribute("key1","ExcelR");

document.write(p.getAttribute("key1"));
</script>

</body>
```

3)removeAttribute(): removeAttribute() is a method in the DOM (Document Object Model) used to remove an attribute from an HTML element. This method deletes the specified attribute from the element, effectively removing any associated value or behavior.

Syntax:

```
element.removeAttribute(attributeName);
```

```
<body>
    <p id="my_p">Hello, ExcelR</p>

<script>

p.setAttribute("key1","ExcelR");

document.write(p.getAttribute("id"));
document.write(p.getAttribute("key1"));
p.removeAttribute("key1");

</script>

</body>
```

4) hasAttribute(): hasAttribute() is a method in the DOM (Document Object Model) used to check whether a specified element has a particular attribute. This method is useful for determining if an element includes a specific attribute before attempting to work with it.

Syntax:

```
let hasAttr =
element.hasAttribute(attributeName);
```

```
<body>
    <p id="my_p">Hello, ExcelR</p>

<script>

   let p =
document.getElementById("my_p");

document.write(p.hasAttribute("id"));

</script>
</body>
```

CSS manipulation

1) classList.add():classList.add() is a method in the DOM (Document Object Model) used to add one or more class names to an element's list of classes. This method is a part of the classList property, which provides a convenient way to manipulate the class attribute of HTML elements.

Syntax:

```
element.classList.add(className1,
        className2, ...);
```

```
<head>
        <title>DOM</title>
        <style>
            .c1{
                color: red;
            }
            .c2{
                color: green;
            }
        </style>
 </head>
<body>
<p id="my_p">Hello,ExcelR</p>
<button
onclick="func_one()">ClickMe</button>
<script>

let p =
document.getElementById("my_p");
  let func_one = ()=>{
    p.classList.add("c1");
      }
</script>

</bodv>
```

2) classList.remove(): classList.remove() is a method in the DOM (Document Object Model) used to remove one or more class names from an element's list of classes. It is part of the classList property, which provides an easy way to manage the class attribute of HTML elements.

Syntax:

```
element.classList.remove(className1,
        className2, ...);
```

```
<head>
        <title>DOM</title>
        <style>
            .c1{
                color: red;
            }
            .c2{
                color: green;
            }
        </style>
 </head>
<body>
<p id="my_p">Hello,ExcelR</p>
<button
onclick="func_one()">ClickMe</button>
<button
onclick="func_two()">Remove</button>

<script>

let p = document.getElementById("my_p");
  let func_one = ()=>{
    p.classList.add("c1");
      }

let func_two = ()=>{
    p.classList.remove("c1");
     }
</script>
</body>
```

3) classList.contains(): classList.contains() is a method in the DOM (Document Object Model) used to check if an element has a specific class. This method is part of the classList property, which provides convenient access to an element's list of classes.

Syntax:

```
let hasClass =
element.classList.contains(className);
```

```
<head>
        <style>
            .c1{
                color: red;
            }
            .c2{
                color: green;
            }
        </style>
 </head>
```

```
<body>
<p id="my_p">Hello,ExcelR</p>
<button
onclick="func_one()">ClickMe</button>
<button
onclick="func_two()">Remove</button>
<button
onclick="func_three()">Check</button>


<script>

let p = document.getElementById("my_p");
  let func_one = ()=>{
    p.classList.add("c1");
      }

let func_two = ()=>{
    p.classList.remove("c1");
     }

let func_three= ()=>{
console.log(p.classList.contains("c1"));
      //true or false
            }
</script>
</body>
```

4) classList.toggle(): classList.toggle() is a method in the DOM (Document Object Model) used to add or remove a class from an element's list of classes, depending on whether the class is already present. This method is a part of the classList property, which simplifies the management of an element's class attribute. (Combination of classList.add() and classList.remove() called as “Toggle”).

Syntax:

```
element.classList.toggle(className, [force]);
```

```
<head>
        <style>
            .c1{
                color: red;
            }
            .c2{
                color: green;
            }
        </style>
 </head>
```

```
<body>
<p id="my_p">Hello,ExcelR</p>
<button
onclick="func_one()">ClickMe</button>
<button
onclick="func_two()">Remove</button>
<button
onclick="func_three()">Toggle</button>

<script>

let p = document.getElementById("my_p");
  let func_one = ()=>{
    p.classList.add("c1");
      }

let func_two = ()=>{
    p.classList.remove("c1");
     }

let func_three = ()=>{
    p.classList.toggle("c1");
       }
</script>
</body>
```

Event Handling

1) addEventListener(): addEventListener() is a method in the DOM (Document Object Model) used to attach event handlers to DOM elements. This method allows you to specify a function that will be executed when a particular event occurs on the element.

Syntax:

```
element.addEventListener(eventType,
eventHandler);
```

```
<body>
<button id="my_btn">Button1</button>
<script>

let btn =
document.getElementById("my_btn");
btn.addEventListener("click",func_one);
 function func_one(){
console.log("Hello");            //Hello
             }
</script>
</body>
```

2)removeEventListener():

removeEventListener() is a method in the DOM (Document Object Model) used to remove an event listener that was previously added to an element using addEventListener(). This method allows you to stop handling a particular event type on an element by removing the associated event handler.

Syntax:

```
element.removeEventListener(eventType,
eventHandler);
```

```
<body>
<button id="my_btn2">Button2</button>
<script>

let btn2 =
document.getElementById("my_btn2");
btn2.addEventListener("click",func_two);
  setTimeout(function(){
btn2.removeEventListener("click",func_two
);
 },2000);
   function func_two(){
   console.log("Hello");        //Hello
       }
</script>
</body>
```

3) dispatchEvent(): dispatchEvent() is a method in the DOM (Document Object Model) used to programmatically trigger an event on a DOM element. This method allows you to create and dispatch custom events or simulate standard events, making it possible to programmatically interact with event-driven behaviors in your application.

Syntax:

```
element.dispatchEvent(event);
```

```
<body>
 <button id="my_btn3">Button3</button>
<script>
```

```
let btn3 =
document.getElementById("my_btn3");
function handleClick(event) {
 alert('Button was clicked
programmatically!');
    }
  btn3.addEventListener('click',
handleClick);
  let clickEvent = new Event('click');
  btn3.dispatchEvent(clickEvent);
        </script>
    </body>
```

Q) Write a program so that whenever the login button is clicked, a success message is shown in green color if the login is successful; otherwise, a red-colored message is displayed, using DOM manipulation.

```
<!DOCTYPE html>
<html>
    <head>
        <title>Login</title>
        <style>
            #message {
                display: none;
                margin-top: 20px;
                font-size: 16px;
                padding: 10px;
                border-radius: 5px;
            }
            .success {
                color: white;
                background-color: green;
            }
            .failure {
                color: white;
                background-color: red;
            }

        </style>
    </head>
<body>
   <div>
<label for="username">Username:</label>
<input type="text" id="username">
   </div>
   <div>
<label for="password">Password:</label>
<input type="password" id="password">
   </div>
```

```
<button id="loginButton">Login</button>

<div id="message"></div>
<script>
document.getElementById('loginButton').
addEventListener('click', function() {

// Get the values of the username and
password inputs
 let username =
document.getElementById('username').val
ue;
 let password =
document.getElementById('password').val
ue;

// Get the message element
let msg =
document.getElementById('message');

// Check if the login is successful
(this is just a simple condition)
  if (username === 'ExcelR' && password
=== 'ExcelR@123') {
// Success: Show a success message in
green
msg.textContent = 'Login successful!';
msg.className = 'success';
} else {
// Failure: Show a failure message in
red
msg.textContent = 'Login failed. Please
try again.';
msg.className = 'failure';
}

// Display the message element
msg.style.display = 'inline-block';

});
        </script>
    </body>
</html>
```